Практическая работа № 2.1 Разработка паттерна MVC на PHP.

Цель: Оптимизация разработки при помощи архитектурных паттернов разработки.

Теория:

Model, View, Controller (MVC) — это шаблон (паттерн) программирования, разделяющий архитектуру приложения на три модуля: модель (Model), представление (View), контроллер (Controller). Простыми словами, он позволяет изменять каждый компонент независимо друг от друга для простой разработки и поддержки веб-приложений.

Модель (Model)

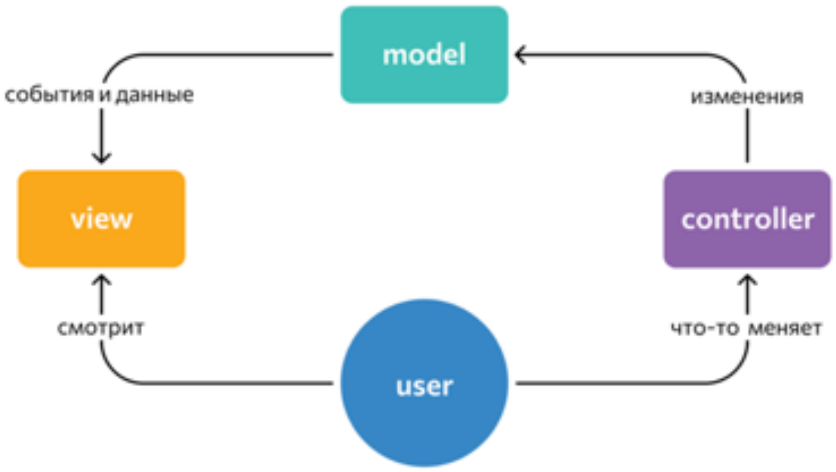
Это основная логика приложения. Отвечает за данные, методы работы с ними и структуру программы. Модель реагирует на команды из контроллера и выдает информацию и/или изменяет свое состояние. Она передает данные в представление.

Представление (View)

Задача компонента — визуализация информации, которую он получает от модели. View отображает данные на уровне пользовательского интерфейса. Например, в виде таблицы или списка. Представление определяет внешний вид приложения и способы взаимодействия с ним.

Контроллер (Controller)

Он обеспечивает взаимодействие с системой: обрабатывает действия пользователя, проверяет полученную информацию и передает ее модели. Контроллер определяет, как приложение будет реагировать на действия пользователя. Также контроллер может отвечать за фильтрацию данных и авторизацию.



Есть вариации схемы. Например, в некоторых пользователь взаимодействует с контроллером не напрямую, а через интерфейс. Но основной принцип работы шаблона остается тем же.

Компоненты модели различаются степенью зависимости друг от друга и ограничениями:

- модель не зависит от представления и контроллера, но и не может использовать классы из их разделов;
- представление может обращаться к модели за данными и событиями, но не может ее менять;
- контроллер не может отображать данные, но способен менять модель в зависимости от действий пользователя.

Концепцию MVC предложил в конце 1970-х годов сотрудник Xerox Трюгве Реекскауг. Она была реализована в языке программирования Smalltalk-80. Окончательную версию шаблона опубликовали только 10 лет спустя в журнале Technology Object. Концепция стала популярна с появлением быстро развертываемых фреймворков и интерактивных веб-приложений.

Как работает MVC

Разберем на реальном примере. Условная физическая модель MVC-архитектуры — персональный компьютер, в котором:

• контроллер — клавиатура или мышь. С их помощью пользователь вводит команды;

• модель — системный блок, в котором происходит обработка команд и хранятся системные и пользовательские файлы;

• представление — монитор, на котором визуализируется работа системного блока.

На этом примере легче понять зависимость компонентов друг от друга:

• если заменить системный блок (без переноса старых данных), но оставить старый монитор и клавиатуру, пользователь получит другой компьютер с новой ОС, файлами, системными характеристиками, драйверами и т.д.;

• если поменять монитор и клавиатуру, но оставить старый системный блок, человек будет использовать прежнюю ОС, файлы, системные характеристики. Изменения коснутся только некоторых драйверов.

В архитектуре одного приложения может быть несколько представлений. Например, в Excel одни и те же данные выводятся в виде круговой диаграммы или таблицы. В некоторых играх можно переключить вид от первого лица на вид от третьего или вообще перейти в режим карты. Пример из веб-разработки — мобильный и десктопный интерфейс сайтов и приложений. Все это разные варианты View для одной и той же Model.

Практика:

Задание 1.

Создайте **index.php**

```php
<?php

declare(strict_types=1);

// Что вообще пришло?
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$uri = rtrim($uri, '/') ?: '/';

echo "<pre>";
echo "REQUEST_URI: {$_SERVER['REQUEST_URI']}\n";
echo "PATH:        {$uri}\n";
echo "METHOD:      {$_SERVER['REQUEST_METHOD']}\n";
echo "</pre>";

// Простейшая маршрутизация вручную
switch ($uri) {
    case '/':
        echo "<h1>Главная страница — список статей</h1>";
        break;

    case '/about':
        echo "<h1>О проекте</h1>";
        break;
```

```
    default:
        if (preg_match('#^/article/(\d+)$#', $uri, $m)) {
            echo "<h1>Статья №{$m[1]}</h1>";
        } else {
            http_response_code(404);
            echo "<h1>404 — страница не найдена</h1>";
        }
}
```

Создай **.htaccess**

```
apache
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php [QSA,L]
```

Теоретические вопросы:

1) Открой /, /about, /article/5, /article/abc, /nope . Куда идут все запросы ?

Все запросы сначала попадают в index.php, потому что .htaccess перенаправляет запросы на него, если это не существующий файл или папка. Далее index.php анализирует URL и определяет, какой ответ вернуть.

- / → главная страница — список статей;
- /about → страница «О проекте»;
- /article/5 → статья №5;
- /article/abc → 404, так как ID статьи должен состоять только из цифр;
- /nope → 404, так как такого маршрута нет.

2) Что будет, если маршрутов станет 50? А если добавим POST?

Если маршрутов станет 50, код с большим количеством case станет неудобным для чтения, поддержки и изменения. Поэтому лучше использовать отдельный роутер, в котором маршруты хранятся в массиве.

При добавлении POST необходимо учитывать HTTP-метод запроса и создавать отдельную обработку POST-маршрутов. В представленном роутере для этого используется метод post().

Задание 2.

Роутер (Router, маршрутизатор) в архитектуре MVC — это компонент, который принимает входящий HTTP-запрос (URL) и определяет, какому контроллеру и какому методу (действию) этого контроллера его передать. **Создаем app/Core/Router.php**

```
<?php

namespace App\Core;

class Router
{
    private array $routes = [];

    public function get(string $path, callable|array $handler): void
    {
        $this->routes['GET'][$path] = $handler;
    }

    public function post(string $path, callable|array $handler): void
    {
```

```
        $this->routes['POST'][$path] = $handler;
    }

    public function dispatch(string $method, string $uri): void
    {
        $uri = parse_url($uri, PHP_URL_PATH);
        $uri = rtrim($uri, '/') ?: '/';

        foreach ($this->routes[$method] ?? [] as $route => $handler) {
            $pattern = '#^' . preg_replace('#\{(\w+)\}#', '(\d+)', $route) . '$#';

            if (preg_match($pattern, $uri, $matches)) {
                array_shift($matches);

                if (is_array($handler)) {
                    [$class, $action] = $handler;
                    (new $class())->$action(...$matches);
                } else {
                    $handler(...$matches);
                }
                return;
            }
        }

        http_response_code(404);
        echo '404 Not Found';
    }
}
```

index.php (переписываем)

```php
<?php

declare(strict_types=1);

spl_autoload_register(function (string $class) {
    $prefix = 'App\\';
    $baseDir = __DIR__ . '/app/';

    if (!str_starts_with($class, $prefix)) return;

    $file = $baseDir . str_replace('\\', '/', substr($class, strlen($prefix))) . '.php';
    if (file_exists($file)) require $file;
});

use App\Core\Router;

$router = new Router();

// Пока — замыкания, контроллеров ещё нет
$router->get('/', fn() => print '<h1>Главная — список статей</h1>');
$router->get('/about', fn() => print '<h1>О проекте</h1>');
$router->get('/article/{id}', fn($id) => print "<h1>Статья №{$id}</h1>");

$router->dispatch($_SERVER['REQUEST_METHOD'],
$_SERVER['REQUEST_URI']);
```

Теоретические вопросы:

- Что изменилось?

Маршрутизация была вынесена из index.php в отдельный класс Router. Теперь маршруты регистрируются через методы get() и post(), а их обработкой занимается роутер.

- Что делает роутер?

Роутер принимает HTTP-метод и URL запроса, сравнивает их с зарегистрированными маршрутами и определяет, какой обработчик необходимо вызвать. Если подходящий маршрут не найден, возвращается ошибка 404 Not Found.

- Кто должен решать, что делать на /article/{id}?

В данном примере решение принимает обработчик, указанный при регистрации маршрута:

```
$router->get('/article/{id}', fn($id) => print "<h1>Статья №{$id}</h1>»);
```

В полноценной MVC-архитектуре эту задачу должен выполнять **контроллер**: роутер определяет нужный контроллер и его метод, а контроллер уже решает, что делать с полученным id.

- Что храниться в массиве $routes и что в нем храниться ?

В $routes хранятся зарегистрированные маршруты, разделённые по HTTP-методам (GET, POST и т. д.). Для каждого маршрута хранится путь и обработчик (callable или массив с классом контроллера и названием его метода). Таким образом, $routes содержит связь между URL-маршрутом и кодом, который должен его обработать.

ЛИТЕРАТУРА

- https://blog.skillfactory.ru/glossary/mvc/ - Что такое MVC.
- https://www.php.net/manual/ru/ - Официальная русская документация по PHP.

https://metanit.com/php/tutorial- Учебник по PHP.